

消息队列

一、什么是消息队列

在计算机科学中，消息队列（英语：Message queue）是一种进程间通信或同一进程的不同线程间的通信方式，软件的队列用来处理一系列的输入，通常是来自用户。消息队列提供了异步的通信协议，每一个队列中的纪录包含详细说明了资料，包含发生的时间，输入设备的种类，以及特定的输入参数，也就是说：消息的发送者和接收者不需要同时与消息队列交互。消息会保存在队列中，直到接收者取回它。

消息队列（Message Queue），是分布式系统中重要的组件，其通用的使用场景可以简单地描述为，当不需要立即获得结果，但是并发量又需要进行控制的时候，差不多就是需要使用消息队列的时候。

二、为什么使用消息队列？

解耦

看这么个场景。A 系统发送数据到 BCD 三个系统，通过接口调用发送。如果 E 系统也要这个数据呢？或者 B 系统现在不需要呢？那么 A 系统就要因为依赖方的原因去做改动适配。而如果使用 MQ，那么 A 系统不用做任何改动，下游依赖业务自己决定接入消息。

异步

A 系统接收一个请求，需要在自己本地写库，还需要在 BCD 三个系统写库，自己本地写库要 3ms，BCD 三个系统分别写库要 300ms、450ms、200ms。最终请求总延时是 $3 + 300 + 450 + 200 = 953\text{ms}$ ，接近 1s，但是如果用 MQ，A 系统只需要发送消息给 BCD，那么耗时就很低。

削峰

在一些高并发大流量场景，短时间大量请求过来，打到 db，可能导致 db 直接崩溃。如果使用 mq，请求挤压到消息队列，由下游业务稳定消费，确保系统正常运行。

三、常见消息队列对比

		kafka	RocketMQ	RabbitMQ	qmq
应用场景	应用场景	系统间的数据流管道，试试数据处理 例如：常规的消息系统、网站火星跟踪、日志收集处理等	非日志性的可靠性消息传输 例如：订单、交易、充值	可靠消息传输，和RocketMQ类似	可靠消息传输，和RocketMQ类似
基础对比	成熟度	日志领域成熟	成熟	成熟	成熟
	所属社区/公司	Apache	ali开发，加入Apache	Mozilla public license	trip/开源
	社区活跃度	高	高	高	一般
	文档完备性	高	高	高	一般
	开发语言	scala	java	erlang	java
	支持协议	自定义	自定义	AMQP	自定义
可用性、可靠性对比	持久化方式	磁盘文件	磁盘文件	内存、文件	磁盘文件
	部署方式	单机/集群	单机/集群	单机/集群	单机/集群
	选主方式	从ISR中自动选一个leader	name server		
	可用性	非常高 (分布式、主从)	非常高 (分布式、主从)	高 (主从，采用镜像模式时间，数据量大时可能产生性能瓶颈)	非常高 (分布式、主从)
	主从切换	自动切换 N个副本，永撑n-1个失效，master失效后自动从ISR中选择一个主	不支持自动切换 master失效后不能向master发送信息，consumer大概30秒可以感知此时间，此后从slave消费；如果master无法恢复，异步复制时可能出现部分信息丢失	自动切换 最早加入集群的slave会成为master，因为新加入的slave不会同步master之前的数据，所以可能会出现部分数据丢失	不支持自动切换 目前当master离线后，不提供自动切换功能，需要人工启动master，当slave离线后，该group不再提供接收消息服务，只提供消息消费服务。当master出现故障，导致消息丢失时，可以将其切换为slave，原来的slave切换为master，slave将从master同步数据，同步完成后提供服务。
	数据可靠性	很好 支持producer单条发送，同步刷盘，同步复制，但这样性能明显下降	很好 支持produce单条发送，broker端支持同步刷盘、异步刷盘，同步双写，异步复制	好 producer支持同步/异步ack，支持列数据持久化，镜像模式中支持主从同步	好 QMQ使用的是异步刷盘，在配置文件里可以配置刷盘的间隔，QMQ依赖主从同步来提高可靠性，在生产环境上如果配置了从，那么MASTER收到消
功能对比	消息写入性能	非常好 每条10字节测试：百万条/s	很好 每条10字节测试：单机单broker 7w/s 单机3broker约为 12w/s	RAM约为rocketMq的1/2 Disk的性能约为Ram性能的1/3	类似rocketMq
	性能稳定性	队列/分区多事性能不稳定，明显下降，消息堆积时性能稳定	队列较多，消息堆积时性能稳定	消息堆积时，性能不稳定，明显下降	消息堆积时性能稳定
	堆积能力	非常好 消息存储在log中，每个分区一个log文件	非常好 所有消息存储在同一个commit log中	一般 生产者、消费者正常时，性能表现稳定；消费者不消费时，性能不稳定	非常好 消息存储在log中，每个分区一个log文件
	复制备份	消息先写入leader的log，follower从leader中pull到数据以后，先ack leader，然后写入log中，ISR中维护于leader同步的列表，落后太多的follower被删除掉	同步双写 异步复制：slave自动线程从master拉数据	普通模式不复制 镜像模式：消息先到master，然后写到slave上，入集群之前的消息不会被复制到新的slave上	同步复制 只有消息同步到slave后master才会返回成功的响应给producer，这就保证了master和slave上都有一致的消息
	消息投递实时性	毫秒级 具体由consumer轮训建个时间决定	毫秒级 支持pull、push两种模式，延时通常在毫秒级	毫秒级	毫秒级 支持pull、push两种模式，延时通常在毫秒级
	顺序消息	支持顺序消费 (但是一台broker宕机后，就会产生消息乱序)	支持顺序消费 在顺序消息场景下，消费失败时消费队列将会暂停	支持顺序消费 但是如果一个消费失败，次消息顺序会被打乱	支持顺序消息
功能对比	定时消息	不支持	开源版本仅支持定时level	不支持	支持 延时/定时消息的精度在1秒左右
	事务消息	不支持	支持	不支持	支持
	broker端消息过滤	不支持	支持 通过tag过滤，类似于子topic	不支持	支持 通过tag过滤，类似于子topic
	消息查询	不支持	支持 根据messageId查询 根据messageKey查询	不支持	messageId查询 制定时间范围查询
	消息失败重试	不支持失败重试	支持失败重试，offset存储在broker中	支持失败重试	
	消息重新消费	支持通过修改offset来重新消费	支持按照时间来重新消费		
	发送端负载均衡	可自由制定	可自由制定	需要单独的loadbalancer支持	可配置负载均衡策略
	消费并行度	消费并行度和分区数一致	顺序消费：消费并行度和分区数一致 乱序消费：消费服务器和消费线程数之和	镜像模式下其实也是master消费	顺序消费：消费并行度和分区数一致 乱序消费：消费服务器和消费线程数之和
	消费方式	consumer pull	consumer pull broker push	broker push	consumer pull broker push
	批量发送	支持 默认producer缓存，压缩，然后批量发送	不支持	不支持	
	消息清理	制定文件保存时间，过期删除	制定文件保存时间，过期删除	可用内存少于40%，触发gc，gc时找到	

二、 携程消息队列框架-QMQ

QMQ 设计背景

现在市面上已经有很多消息中间件了(ActiveMQ, RabbitMQ, Kafka, RocketMQ), 那么为什么我们还要造另外一个轮子呢? 首先QMQ是2012年开始开发的, 在这个时期其实消息中间件并没有太多的选择, 那个时候Kafka还不太成熟,而RocketMQ也没有出现, 大部分公司都会采用ActiveMQ或RabbitMQ. 首先RabbitMQ的开发语言是erlang, 这是一门小众的语言, 我们担心无法完全掌控, 所以没有选择。而ActiveMQ其实在公司内部已有很长一段时间使用历史, 但是ActiveMQ太过于复杂, 在使用过程中经常出现消息丢失或者整个进程hang住的情况, 并且难以定位。

当然, 这都是QMQ诞生的历史背景, 那么放在今天QMQ还有开源的意义吗? 我们比较了市面上现有的消息中间件, 仍然认为QMQ有自己的特色, 而这些也是我们在过去几年运维消息中间件中觉得必须提供的和难以舍弃的。

像 kafka,rocketmq 等, 在设计的时候就是要保证分区下消息的顺序, 也就是说消息在一个分区中的顺序是怎样的, 那么消费者在消费的时候看到的就是什么样, 那么每个分区只能被一个消费组的一消费者消费, 这也导致不能动态扩容缩容。而 qmq 并没有从框架本身保证分区消费的顺序性, 而是通过对有序消息的消费并行度进行限制。

3.1 QMQ 架构

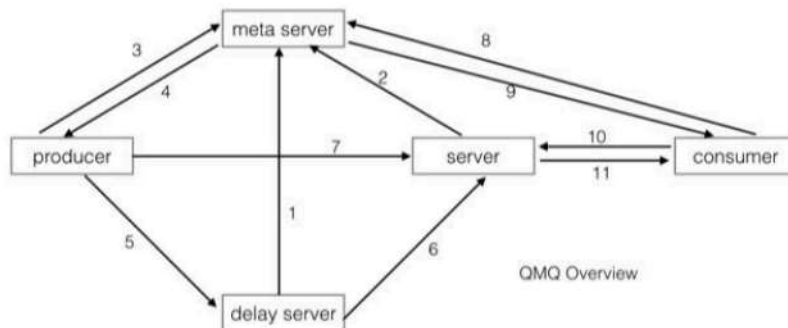
meta server 提供集群管理和集群发现的作用

server 提供实时消息服务

delay server 提供延时/定时消息服务, 延时消息先在 delay server 排队, 时间到之后再发送给 server

producer 消息生产者

consumer 消息消费者



1.delay server 向 meta server 注册

2.实时 server 向 meta server 注册

3.producer 在发送消息前需要询问 meta server 获取 server list

4.meta server 返回 server list 给 producer(根据 producer 请求的消息类型返回不同的 server list)

5.producer 发送延时/定时消息

6.延时时间已到, delay server 将消息投递给实时 server

7.producer 发送实时消息

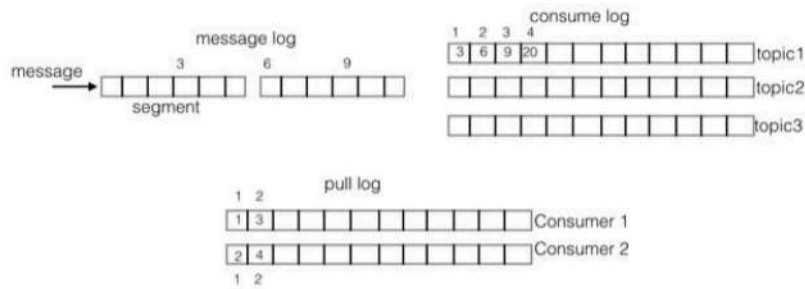
8.consumer 需要拉取消息, 在拉取之前向 meta server 获取 server list(只会获取实时 server 的 list)

9.meta server 返回 server list 给 consumer

10.consumer 向实时 server 发起 pull 请求

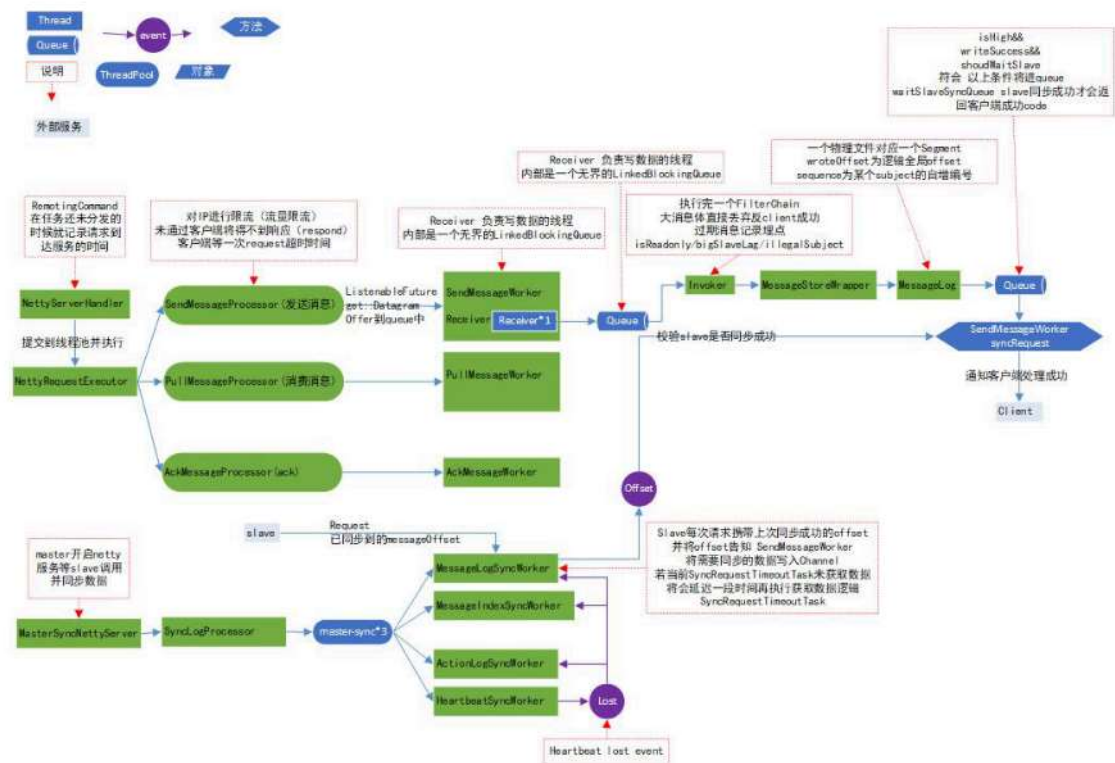
11.实时 server 将消息返回给 consumer

3.2 QMQ 存储模型



- **message log:** 所有 subject 的消息进入该 log, 消息的主存储
- **consume log:** consume log 存储的是 message log 的索引信息, 也就是每个 topic 在 message log 上的索引
- **pull log:** 每个 consumer 拉取消息的时候会产生 pull log, pull log 记录的是拉取的消息在 consume log 中的 sequence

3.3 QMQ 发送消息



3.4 QMQ 高级特性

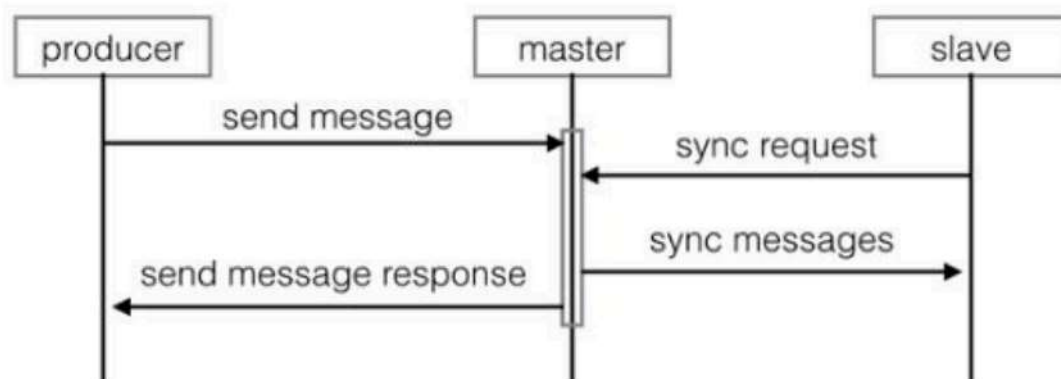
3.4.1 QMQ 之高可用(HA)

QMQ 分别从两个角度提供高可用能力：分片和复制

首先因为 QMQ 不是基于 partition 的，所以很容易通过添加更多的机器就能提高一个 subject 的可用性，消息按照一定的负载均衡策略分布在不同的机器上，某台机器离线后 producer 将不再将消息发送给该 Server。

除此之外，QMQ 通过主从复制来提高单机可用性。QMQ 将服务端集群划分为多个 group，每个 group 包含一个 master 和一个 slave。消息的发送和消费全部指向 master，slave 只为保证可用性。

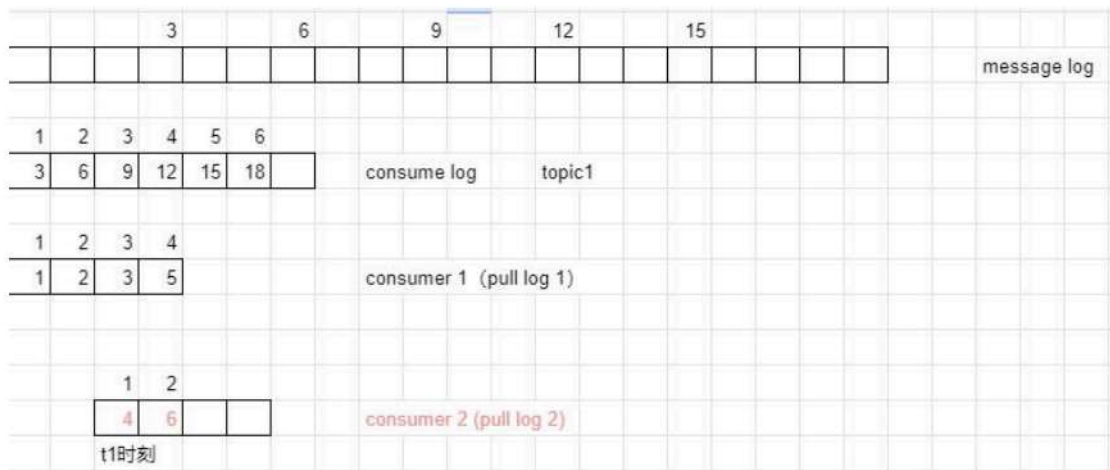
当消息发送给 master 后，slave 会从 master 同步消息，只有消息同步到 slave 后 master 才会返回成功的响应给 producer，这就保证了 master 和 slave 上都有一致的消息。当 master 和 slave 之间的延迟增大时，会标记该 group 为 readonly 状态，这个时候将不再接收消息，只提供消息消费服务。下图为消息发送和主从同步示意图：



目前当 master 离线后，不提供自动切换功能，需要人工启动 master。当 slave 离线后，该 group 不再提供接收消息服务，只提供消息消费服务。当 master 出现故障，导致消息丢失时，可以将其切换为 slave，原来的 slave 切换为 master，slave 将从 master 同步数据，同步完成后提供服务。

3.4.2 QMQ 之 consumer 动态扩容缩容

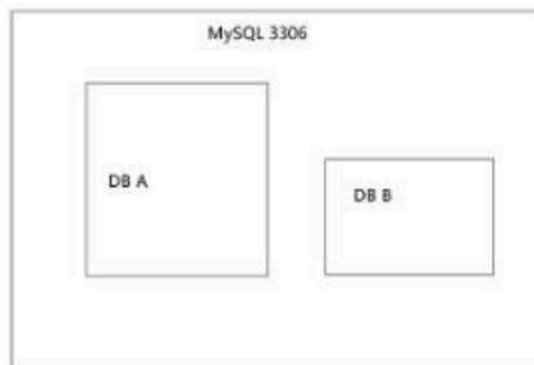
像 kafka,rocketmq,而且框架在设计之初，就规定了一个分区只能被一个消费者消费，所以就不能动态的扩容消费者。而 qmq 引入了 pull log，在 consumer log 之上再建了一层索引，如果需要扩容 consumer，那么 pull log 可以很方便的重新负载均衡 consume log。



在 t1 时刻,业务方扩容了一台 consumer2,通过 pull log, 很容易就将 consume log 分配到两个 consumer 的 pull log 上,从而达到动态扩容的目的。

3.4.3 QMQ 之事务消息

和 rocketMQ 不同的是, QMQ 并没有采用分布式事务 (XA) 等解决方案来实现事务消息,而是巧妙的利用 Mysql 数据库自生的事务支持,实现了事务消息。



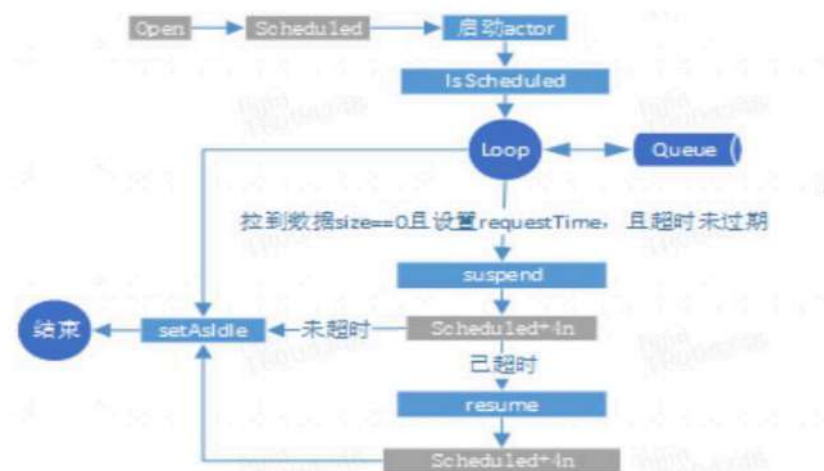
提到一致性,大家肯定就想到事务,而一提到事务,肯定就想到关系型数据库,那么我们是不是可以借助关系型 DB 里久经考验的事务来实现这个一致性呢。我们以 MySQL 为例,对于 MySQL 中同一个实例里面的 db,如果共享相同的 Connection 的话是可以在同一个事务里的。

有了这层保证,我们就可以透明的实现业务操作和消息发送在同一个事务里了,首先我们在公司所有 MySQL 实例里初始化出一个 message db,这个可以放到自动化流程中,对应用透明。然后我们只要将发消息与业务操作放到同一个 DB 事务里即可。

将消息和业务绑定到同一个事务里面保证了消息一定会被记录到 db,这样在事务成功时就可以发送事务消息到 server,如果消息发送失败则重试,保证消息一定可以发出去。

3.4.4 QMQ 之 Actor

QMQ 通过引入类 actor 处理模型的方式，很好的做到的公平调度，即使某个消费组消费者再多，另外一个消费组的消费者再少也能获得自己应有的时间片。QMQ 为每个消费组分配一个 actor，那么来自这个消费组的所有拉请求都会进入对应 actor 的信箱(队列)，然后这些 actor 再被调度。



3.4.5 QMQ 其他问题

待补充